Git Recovery & .gitignore Mastery

How to Revert a Bad Commit and Safely Exclude Large / Sensitive Files

---

1. What Happened (The Problem)

A commit was made that deleted almost every file from the repository tracking (while the files remained on disk as untracked). The remote GitHub repository became nearly empty. This is a common situation when large folders (especially SSAS `bin/` and `obj/` folders) or accidental mass deletions are committed.

Key insight: Files still existing on your computer as "untracked" can be recovered. Git history still contains the previous good commits.

2. Core Concepts We Used

A. Reverting / Resetting to a Good Commit

When a bad commit reaches the remote, the safest recovery is:

- Find the last good commit hash with `git log --oneline`
- Reset the local branch to that commit: `git reset --hard <good-hash>`
- Force-push the corrected history: `git push origin main --force`

B. .gitignore – Preventing Future Problems

`.gitignore` tells Git which files and folders should never be tracked. It is essential for:

- Large data files (CSV, Parquet, Excel, models)
- Build artifacts (`bin/`, `obj/`, `.vs/`)
- Sensitive information (.env, credentials, API keys)
- Temporary / generated files (checkpoints, caches)

Important rule: `.gitignore` only works on *untracked* files. Once a file is committed, you must remove it from tracking with `git rm --cached` before the ignore rule takes effect.

3. Step-by-Step Recovery Process

Step 1 – Inspect the situation

```
git log --oneline -8
git status
dir (or ls)
```

Step 2 – Reset to the last good commit

```
git reset --hard b54ef65
# Replace b54ef65 with your actual good commit hash
```

Step 3 – Force-push the clean history to GitHub

```
git push origin main --force
```

Step 4 – Create a correct .gitignore (in project root)

```
# Visual Studio / SSAS build artifacts
**/bin/
**/obj/
**/.vs/
*.user
*.suo

# Specific SSAS folders
```

```
SQL SCRIPTS/VERSION2/Malaria_SSAS_project/bin/
SQL SCRIPTS/VERSION2/Malaria_SSAS_project/obj/
SQL SCRIPTS/VERSION2/Malaria_SSAS_project/**/bin/
SQL SCRIPTS/VERSION2/Malaria_SSAS_project/**/obj/
```

Step 5 – Add only the source files (never bin/obj)

```
git add .gitignore
git add "Malaria Report/Improvements/Version_with _SSAS.pbix"
git add "SQL SCRIPTS/VERSION2/Malaria_SSAS_project/"
git add "SQL SCRIPTS/VERSION2/Solution1/"
git add "SQL SCRIPTS/VERSION2/Solution2/"
git status
```

Step 6 – Commit with a meaningful message

```
git commit -m "Introduce SSAS (OLAP) semantic model as single source of truth

Decoupled the analytics layer from the production database.
Previously Power BI imported views and tables directly.
Now using SSAS for centralized business logic, better performance,
and support for live connection / DirectQuery."
```

Step 7 – Push the clean commit

```
git push origin main
```

4. Correct .gitignore Patterns (Best Practices)

Never use full Windows paths (e.g. `D:\Data Analyst\...`). Git expects relative patterns from the repository root.

Recommended patterns for data & analytics projects:

```
# Large data files
*.csv
*.parquet
*.pkl
*.h5
*.xlsx
*.xls
data/
datasets/

# Python
__pycache__/
*.py[cod]
.venv/
venv/
.env

# Jupyter
.ipynb_checkpoints/

# Visual Studio / SSAS
**/bin/
**/obj/
**/.vs/
*.user
*.suo

# OS / IDE
.DS_Store
Thumbs.db
.idea/
.vscode/
```

5. Essential Commands Cheatsheet

Command	Purpose
`git log --oneline -10`	See recent commits and their hashes
`git status`	See staged, unstaged and untracked files
`git reset --hard <hash>`	Move branch + working tree to a previous commit
`git push origin main --force`	Overwrite remote history (use carefully)
`git rm -r --cached <folder>`	Stop tracking a folder but keep it on disk
`git add .gitignore`	Start tracking the ignore rules
`git commit -m "message"`	Create a new commit
`git push origin main`	Publish the commit to GitHub

6. Lessons Learned

- **Never commit build artifacts** (`bin/`, `obj/`). They cause “Filename too long” errors and bloat the repository.
- **Always check** `git status` before committing. Staged deletions of many folders is a red flag.
- **`.gitignore` is your friend** for large data files, secrets, and generated folders. Create it early.
- **Force-push is powerful but dangerous**. Only use it when you are sure you are restoring a known-good state and no one else is collaborating on the same branch.
- **Files on disk are not lost** just because Git shows them as deleted or untracked. Recovery is usually possible.
- **Meaningful commit messages** document architectural decisions (e.g. why you introduced an SSAS semantic model).

7. Why We Introduced SSAS (Bonus Context)

Previously Power BI connected directly to warehouse views and tables. We moved to a dedicated SSAS (OLAP) semantic model so that:

- Business logic and measures live in one place (single source of truth)
- The analytics layer is decoupled from the production OLTP database
- Live connection / DirectQuery become practical and performant
- Future tools (Power BI, Excel, other clients) can reuse the same model

---